

Estudo Completo: Integração Mercado Pago + Ngrok + Cookies

O Problema Original

Ao autorizar o app Kalles ERP no Mercado Pago, o redirect de volta para `/admin/pagamentos/mp-callback` caía na tela de `/login`. O sistema não finalizava a vinculação da conta.

1. Conceitos Fundamentais

🍪 Cookies HTTP (Set-Cookie)

Cookies são pequenos dados que o **servidor** manda para o **navegador** guardar. Em cada request futuro, o navegador reenvia o cookie automaticamente.

No Kalles, o Spring Boot cria o cookie `kalles_auth_token` (um JWT) quando o usuário faz login ou verifica o email:

```
// AuthController.java
ResponseCookie.from("kalles_auth_token", token)
    .httpOnly(true)    // JS do browser NÃO pode ler (proteção contra XSS)
    .secure(false)     // Aceita HTTP (true = só HTTPS)
    .path("/")         // Válido para todas as rotas
    .maxAge(Duration.ofHours(12))
    .sameSite("Lax")   // Política de envio cross-site
    .build();
```

Atributos importantes:

Atributo	O que faz	Valor no Kalles
<code>httpOnly</code>	Impede acesso via JavaScript (<code>document.cookie</code>)	<code>true</code> → proteção contra XSS
<code>secure</code>	Cookie só é enviado em conexões HTTPS	<code>false</code> local, <code>true</code> em produção
<code>path</code>	Em quais caminhos o cookie é enviado	<code>/</code> → todas as rotas
<code>maxAge</code>	Tempo de vida em segundos	12 horas
<code>sameSite</code>	Quando enviar em requests cross-site	<code>Lax</code>

🔒 SameSite (Lax vs Strict)

Controla quando o navegador envia o cookie em requests que **vêm de outro site**.

- **Strict**: NUNCA envia o cookie se o usuário veio de outro site. Problemático para OAuth porque o Mercado Pago redireciona de `mercadopago.com.br` → seu site.

- **Lax** (recomendado): Envia o cookie em **navegações top-level seguras** (cliques em links, redirects GET). Bloqueia em `fetch()`, `<iframe>`, `<img>` cross-site. Perfeito para OAuth.
- **None**: Sempre envia (requer `Secure=true`). Menos seguro.

🌐 CORS (Cross-Origin Resource Sharing)

Quando o browser em `https://ngrok.app` tenta chamar `http://localhost:8080`, ele envia um header `Origin: https://ngrok.app`. O backend precisa responder com `Access-Control-Allow-Origin: https://ngrok.app`, senão o browser bloqueia a resposta.

```
// SecurityConfig.java
@Value("${cors.allowed-origin:http://localhost:3000}")
private String allowedOrigin;

// Configura quais origens podem chamar o backend
configuration.setAllowedOrigins(List.of(allowedOrigin));
configuration.setAllowCredentials(true); // Necessário para cookies!
```

Adicionamos no `application-local.yml`:

```
cors:
  allowed-origin: "https://42f7-...ngrok-free.app"
```

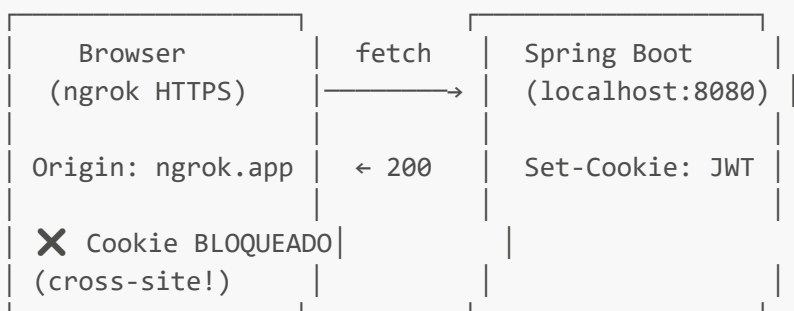
🔒 Mixed Content

Quando uma página HTTPS (`https://ngrok.app`) tenta fazer `fetch("http://localhost:8080")`, o browser **bloqueia silenciosamente**. HTTPS não pode chamar HTTP — é como tentar entrar num cofre com a porta aberta.

🏠 Domínio e Cookies

Cookies são **isolados por domínio**. Um cookie criado em `localhost` **não existe** para `ngrok-free.app`. São mundos separados para o browser.

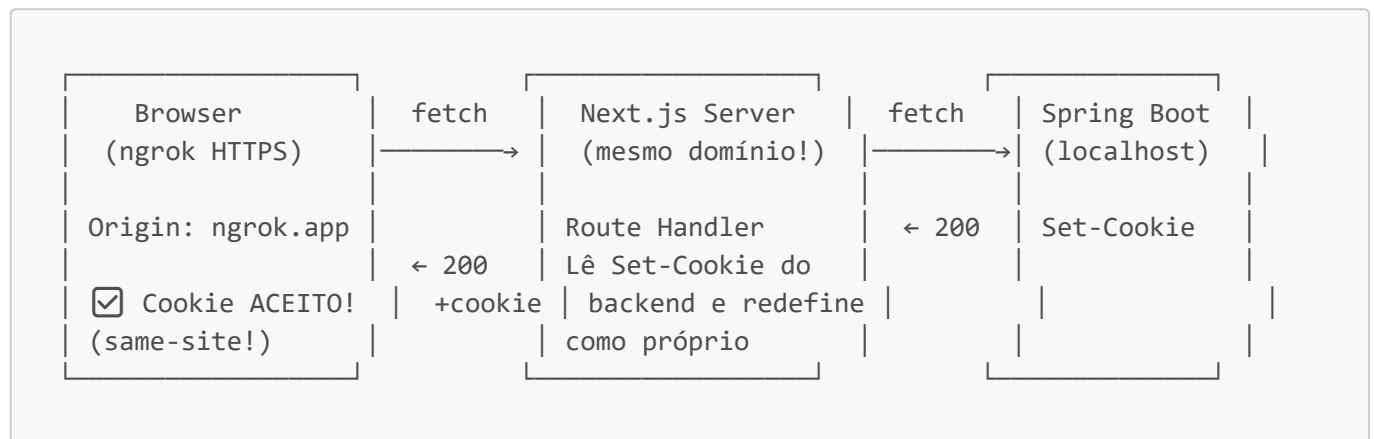
2. Arquitetura ANTES (Quebrada)



Por que falhava:

1. `.env.local` tinha `NEXT_PUBLIC_API_URL=http://localhost:8080`
2. O `api.ts` do frontend mandava TODOS os requests diretamente para o Spring Boot
3. O browser estava no domínio do Ngrok (HTTPS) mas chamava o localhost (HTTP)
4. Chrome bloqueava o `Set-Cookie` como "cross-site response"
5. Sem cookie → middleware do Next.js redirecionava para `/login`

3. Arquitetura DEPOIS (Funcionando) — Padrão BFF



BFF = Backend For Frontend: O Next.js atua como intermediário. O browser nunca fala diretamente com o Spring Boot. Vantagens:

- Cookie vem do **mesmo domínio** (ngrok → ngrok)
- Spring Boot pode ficar em rede privada em produção
- Sem problemas de CORS ou Mixed Content

4. Arquivos e Seus Papéis

Frontend (Next.js)

[.env.local](file:///c:/Users/Carlo/OneDrive/Documentos/kalles/frontend/.env.local)

O que é: Arquivo de variáveis de ambiente LOCAL do Next.js. Nunca vai pro Git.

O que causava: `NEXT_PUBLIC_API_URL=http://localhost:8080` era injetado pelo Next.js em **tempo de compilação** em todo código que usasse `process.env.NEXT_PUBLIC_API_URL`. O prefixo `NEXT_PUBLIC_` faz a variável ser exposta ao browser (lado do cliente).

[!CAUTION] Variáveis `NEXT_PUBLIC_*` são "cozidas" no JavaScript do cliente durante o build. Mesmo que você mude o valor depois, o bundle antigo continua com o valor velho até recompilar.

[api.ts](file:///c:/Users/Carlo/OneDrive/Documentos/kalles/frontend/shared/services/api.ts)

O que é: Serviço centralizado de HTTP do frontend.

```
const BASE_URL = process.env.NEXT_PUBLIC_API_URL ?? "";
```

- Com `.env.local` vazio: `BASE_URL = ""` → requests vão para o **mesmo servidor** (Next.js)
- Com `.env.local = "http://localhost:8080"`: requests iam **direto** para o Spring Boot (causava o problema)

O `credentials: "include"` garante que cookies são enviados junto com cada request.

[middleware.ts] (file:///c:/Users/Carlo/OneDrive/Documentos/kalles/frontend/middleware.ts)

O que é: Interceptador que roda em TODA requisição de página antes de renderizar.

```
// Se não tem cookie E não é página pública → manda para /login
if (!token && !isPublicPage) {
  return NextResponse.redirect(new URL("/login", request.url));
}
```

Matcher: O regex no `config.matcher` exclui `/api`, `_next/static`, etc. Então o middleware NÃO intercepta chamadas de API — só páginas.

[!NOTE] No Next.js 16 o arquivo `middleware.ts` foi renomeado para `proxy.ts`. Ainda funciona, mas mostra um warning.

[route.ts]

(file:///c:/Users/Carlo/OneDrive/Documentos/kalles/frontend/app/api/auth/%5B...path%5D/route.ts)
(Route Handler BFF)

O que é: Intercepta chamadas para `/api/auth/*` no lado do **servidor** do Next.js.

Caminho: `app/api/auth/[...path]/route.ts`

- `[...path]` = catch-all dinâmico (captura `login`, `register`, `verify`, etc.)

Fluxo interno:

1. Browser chama `POST /api/auth/login` (mesmo domínio)
2. Route Handler recebe o request
3. Faz `fetch("http://localhost:8080/api/auth/login")` **server-to-server** (sem browser no meio)
4. Lê o `Set-Cookie` da resposta do Spring Boot
5. Recria o cookie usando `response.cookies.set()` do Next.js
6. Retorna a resposta para o browser **com o cookie do próprio domínio**

```
// O cookie é definido pelo Next.js, não pelo Spring Boot
response.cookies.set({
  name: "kalles_auth_token",
```

```
value: cookieValue,  
httpOnly: true,  
secure: request.nextUrl.protocol === "https:", // auto-detecta  
sameSite: "lax",  
});
```

[next.config.mjs](file:///c:/Users/Carlo/OneDrive/Documentos/kalles/frontend/next.config.mjs)

O que é: Configuração central do Next.js.

```
async rewrites() {  
  return {  
    afterFiles: [  
      {  
        source: "/api/:path*",  
        destination: "http://localhost:8080/api/:path*",  
      },  
    ],  
  };  
},
```

Rewrites: Proxies invisíveis. O browser pensa que está chamando o Next.js, mas o Next.js repassa para o Spring Boot por trás dos panos.

afterFiles: Controla a prioridade de resolução de rotas:

Ordem de resolução do Next.js:

1. headers
2. redirects
3. middleware/proxy
4. beforeFiles rewrites
5. ★ Rotas do filesystem (Route Handlers, pages) ★
6. afterFiles rewrites ← nosso rewrite está aqui
7. fallback rewrites

Usando **afterFiles**, garantimos que o Route Handler em **app/api/auth/** (etapa 5) é checado **ANTES** do rewrite genérico **/api/:path*** (etapa 6). Assim:

- **/api/auth/login** → Route Handler (BFF com cookie explícito)
- **/api/mercadopago/stores** → Rewrite para Spring Boot

allowedDevOrigins: Diz ao Next.js dev server para aceitar requests vindos do domínio do Ngrok sem bloquear como cross-origin.

Backend (Spring Boot)

[AuthController.java](file:///c:/Users/Carlo/OneDrive/Documentos/kalles/kalles-back/kalles-sale/src/main/java/dev/kalles/sale/security/controller/AuthController.java)

O que é: Controller REST que gerencia login, registro, verificação e logout.

O método `createAuthCookie()` é o que cria o JWT cookie. Os atributos `SameSite("Lax")` e `Secure(false)` são definidos aqui.

[JwtAuthenticationFilter.java](file:///c:/Users/Carlo/OneDrive/Documentos/kalles/kalles-back/kalles-sale/src/main/java/dev/kalles/sale/security/filter/JwtAuthenticationFilter.java)

O que é: Filtro do Spring Security que roda em TODA requisição ao backend.

1. Procura o cookie `kalles_auth_token` no request
 2. Decodifica o JWT
 3. Setta o contexto de autenticação do Spring Security
 4. Setta o contexto do tenant (multi-tenancy)
-

[SecurityConfig.java](file:///c:/Users/Carlo/OneDrive/Documentos/kalles/kalles-back/kalles-sale/src/main/java/dev/kalles/sale/security/config/SecurityConfig.java)

O que é: Configuração central de segurança (CORS, rotas públicas, filtros).

A configuração CORS lê `cors.allowed-origin` do YAML e permite apenas essa origem. Em produção, será o domínio real. Em dev com Ngrok, é a URL do Ngrok.

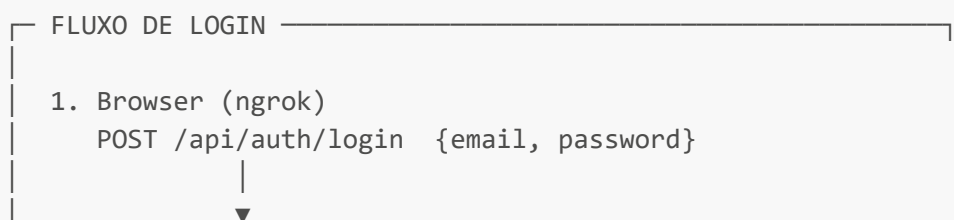
[application-local.yml](file:///c:/Users/Carlo/OneDrive/Documentos/kalles/kalles-back/kalles-sale/src/main/resources/application-local.yml)

O que é: Configurações específicas do ambiente local.

```
cors:
  allowed-origin: "https://42f7-...ngrok-free.app"
```

Sem essa linha, o Spring Boot só aceitava `http://localhost:3000` (valor default do `@Value`).

5. Resumo Visual do Fluxo Completo



2. Next.js Route Handler (app/api/auth/[...path]/route.ts)
→ fetch("http://localhost:8080/api/auth/login")
↓
3. Spring Boot (AuthController.java)
→ Valida credenciais
→ Gera JWT
→ Retorna 200 + Set-Cookie: kalles_auth_token=JWT
↓
4. Route Handler lê o Set-Cookie do Spring Boot
→ Recria o cookie via response.cookies.set()
→ Cookie agora é "do domínio do Next.js/Ngrok"
↓
5. Browser recebe 200 + cookie do MESMO domínio
→ Chrome aceita! Cookie salvo! ☒
↓
6. router.push("/caixas")
→ Middleware checa cookie → ENCONTRADO → permite ☒

FLUXO MERCADO PAGO

1. Usuário clica "Vincular Conta" (ngrok)
→ Vai para auth.mercadopago.com
2. Autoriza o app no Mercado Pago
3. MP redireciona para:
https://ngrok.app/admin/pagamentos/mp-callback?code=X
↓
4. Middleware do Next.js checa cookie
→ Cookie existe (mesmo domínio, SameSite=Lax permite)
→ Permite acesso ☒
↓
5. mp-callback/page.tsx carrega
→ fetch POST /api/v1/mercadopago/oauth/link
→ Backend salva credenciais do vendedor
→ Conta vinculada! 🦊

6. Lições Aprendidas

1. **.env.local é invisível** — não aparece em buscas **find** por ser gitignored, mas o Next.js o carrega silenciosamente

2. **NEXT_PUBLIC_*** é injetado no build — mudar o valor requer restart do `npm run dev`
3. **Cookies são isolados por domínio** — `localhost` ≠ `ngrok-free.app`
4. **Mixed Content é silencioso** — HTTPS chamando HTTP falha sem erro claro
5. **SameSite=Lax é o padrão seguro** — permite OAuth e bloqueia CSRF
6. **BFF (Backend For Frontend)** é o padrão mais seguro para apps web modernas
7. **afterFiles no rewrites** garante que Route Handlers têm prioridade